CMPE 172 · MILESTONE 2

1. System and booking workflow

TutorConnect · Yinqi Chen · October 6, 2026

TutorConnect connects customers with tutors for focused sessions in algebra, physics and writing. Milestone 2 extends the original five-table JDBC skeleton with a usable web interface, authentication, role-based access, booking/cancellation and concurrency protection.

Customers open the available-session list, filter by tutor, service and UTC date, and move through pages. The SQL query excludes inactive slots, past start times and slots with an active appointment; it uses a stable start_time/slot_id ordering plus LIMIT and OFFSET.

After login, a customer opens a booking form for the chosen slot and service. Confirming the form creates a new BOOKED appointment and redirects to an owner-only confirmation page. My appointments separates upcoming sessions from historical/cancelled sessions. Cancellation is permitted only to the owner of a future BOOKED appointment.

Providers add future slots tied to their own services, with duration and overlap validation. They can remove their unreserved availability and view bookings with them. Removal deactivates a slot rather than deleting the historical FK. Recreating an identical removed slot reactivates its existing row. Past BOOKED appointments are marked COMPLETED when the customer/provider appointment list is requested.

Frontend-to-backend request flow

```
Browser form (session cookie + CSRF token)
  -> Spring Security -> PageController
  -> AppointmentService -> BookingTransaction
  -> SlotRepository / AppointmentRepository
  -> JdbcTemplate -> MySQL
  -> DTO / model -> Thymeleaf -> HTML confirmation
```

The browser POSTs slotId and serviceId to /customer/appointments. Security validates the session, CUSTOMER role and CSRF token. The controller obtains the user ID from the authenticated principal and delegates to the service. The repository executes parameterized SQL. On success, the controller redirects to /customer/confirmation/{id}; the confirmation controller checks ownership before returning the Thymeleaf model. JSON clients use /api/customer/appointments and receive HTTP 201 with the appointment ID.

2. Authentication and role-based access

Spring Security loads Account records using a JDBC SELECT against users.email, with email serving as the username. The role and password hash are read from that same database row. BCryptPasswordEncoder verifies credentials; all seeded users have BCrypt hashes rather than plaintext database passwords.

```
SELECT user_id,name,email,password_hash,role
FROM users WHERE email=?
```

On successful login Spring Security establishes a server-side HTTP session. The browser stores only the session cookie, not a JWT or a role chosen by the client. The custom Account principal carries the database user ID and authorities ROLE_CUSTOMER or ROLE_PROVIDER. The session cookie is HttpOnly, uses SameSite=Lax and has a 30-minute session timeout.

```
.requestMatchers("/customer/**", "/api/customer/**")
    .hasRole("CUSTOMER")
.requestMatchers("/provider/**", "/api/provider/**")
    .hasRole("PROVIDER")
```

Security rejects wrong-role requests with HTTP 403. APIs without authentication return 401; page requests redirect to /login. Customer ownership and provider service/slot ownership are also checked in the service layer. Controllers never accept a submitted userId as the actor. All modifying requests require a CSRF token, including booking, cancellation, provider changes and logout.

Validation and safe errors

Services validate positive IDs, matching slot/service, future times, correct duration, valid page/size and provider ownership. Global error handling sends standard statuses and short messages: 400 invalid input; 403 role/ownership; 404 missing record; 409 booking conflict or overlapping time; 503 exhausted retries or a busy database. Unexpected errors are logged on the server and return a generic 500 message; client responses omit stack traces.

3. Race condition and transaction implementation

Without locking, customers A and B can both read an available slot, then both attempt INSERT. A browser-visible availability check is therefore insufficient: the slot must be checked again while the database protects the write.

BookingTransaction.book is a separate Spring-managed bean with @Transactional and READ_COMMITTED isolation. It first locks the existing availability row by primary key using SELECT FOR UPDATE. It verifies the slot is active and in the future, matches the chosen service and has no active booking, then inserts. The lock is held until commit/rollback.

```
SELECT s.*,
  (SELECT p.user_id FROM services sv
   JOIN providers p ON p.provider_id=sv.provider_id
   WHERE sv.service_id=s.service_id) AS provider_user_id
FROM availability_slots s
WHERE s.slot_id=? FOR UPDATE

@Transactional(isolation = Isolation.READ_COMMITTED)
  public long book(long user, long slotId, long serviceId) {
    if (user <= 0 || slotId <= 0 || serviceId <= 0)
      throw new AppException(BAD_REQUEST, "IDs must be positive.");
    var slot =
        slots
            .lock(slotId)
            .filter(SlotRepository.LockedSlot::active)
            .orElseThrow(() -> new AppException(NOT_FOUND, "Slot not found."));
    if (slot.serviceId() != serviceId)
      throw new AppException(BAD_REQUEST, "The selected service does not match this slot.");
    if (!slot.startTime().isAfter(LocalDateTime.now(ZoneOffset.UTC)))
      throw new AppException(CONFLICT, "This slot has already started.");
    if (appointments.occupied(slotId))
      throw new AppException(CONFLICT, "This slot has just been booked. Please choose another.");
    try {
      return appointments.insert(user, slotId);
    } catch (DuplicateKeyException e) {
      throw new AppException(CONFLICT, "This slot has just been booked. Please choose another.");
    }
  }
```

If A acquires the row first, B waits. After A commits, B sees the active appointment and returns 409. If A fails and rolls back, B may succeed. Cancellation and removal acquire that same slot lock before changing its state. Provider creation uses a provider-row lock to serialize overlap checks. No JVM-only synchronized lock is relied upon, so the booking protection works across application processes sharing MySQL.

4. ACID, isolation, uniqueness and retry

ACID applied to one booking

Atomicity: failure rolls the transactional insert back. TransactionTests forces an exception after insertion and verifies that the booking row does not remain. Consistency: foreign keys preserve account/service/slot references, CHECK constraints validate times and statuses, and a unique active-slot key prevents duplicate reservations. Isolation: READ_COMMITTED prevents dirty reads, while the explicit slot row lock serializes conflicting booking/cancellation/removal operations. Durability: MySQL InnoDB stores committed data; Compose mounts a persistent volume. The optional H2 demo profile deliberately discards data on restart.

Chosen isolation level

READ_COMMITTED is sufficient for this implementation because availability is checked after acquiring the slot lock and conflicting writes follow the same locking protocol. A read-only listing can change before the user confirms; the transactional recheck handles that normally as 409. Isolation alone is not claimed to prevent double booking.

Database slot backstop and cancellation history

```
active_slot_id BIGINT GENERATED ALWAYS AS
  (CASE WHEN status IN ('BOOKED','COMPLETED')
   THEN slot_id ELSE NULL END) STORED,
CONSTRAINT uq_appointment_active_slot UNIQUE(active_slot_id)
```

slot_id remains the historical reference. BOOKED and COMPLETED records retain a unique active_slot_id. CANCELLED records produce NULL, so several historical cancellations can coexist while only one current appointment occupies a slot. This preserves the slot uniqueness backstop without the Milestone 1 problem where UNIQUE(slot_id) prevented rebooking forever. A cancelled booking remains with its original customer; a new reservation gets a new appointment ID.

Retry boundary

AppointmentService calls the transactional bean through the Spring proxy. It retries transient pessimistic lock failures at most three times, waiting 25 ms and then 50 ms before the next attempt. Each attempt starts a fresh transaction. Interrupted retries restore the thread interrupt flag. Business conflicts and duplicate-key conflicts are not retried; exhausted lock retries return 503. Provider operations use safe 503 responses for lock failures rather than automatic retries.

5. Verification, operation and deliverables

Validation completed on October 6, 2026 (America/Los_Angeles). The same 31-test suite passed with H2 and with an isolated local MySQL 9.2 instance: 0 failures, 0 errors, 0 skipped in each run. Maven verify also produced the executable Spring Boot JAR. GitHub Actions is configured for Java 17 and MySQL 8.4; those hosted results must be checked separately.

Test class	Count	Behavior
BookingServiceTests	16	Rules, ownership, validation and retry
BookingWorkflowTests	11	HTTP/session/CSRF, UI, provider and race
TutorConnectApplicationTests	2	Database overview and available-slot API
TransactionTests	2	Rollback and direct unique-key backstop

Two-thread concurrency test

Two HTTP clients log in as different customers and keep independent session cookies and CSRF tokens. Two worker threads count down a readiness latch and wait on a shared start latch, then POST the same slot/service. The test waits for both bounded futures, asserts the two responses are exactly 201 and 409 in either order, and asserts exactly one active appointment for that slot in the database. The test is automated and exercises Security, MVC, service proxies, JDBC and the database together.

```
assertThat(List.of(a.get(15, SECONDS), b.get(15, SECONDS)))
    .containsExactlyInAnyOrder(201, 409);
assertThat(jdbc.queryForObject(
    "SELECT COUNT(*) FROM appointments WHERE active_slot_id=1",
    Integer.class)).isEqualTo(1);
```

Additional tests verify cancellation/rebooking history, unauthenticated and wrong-role responses, invalid CSRF, bad login and logout, provider ownership, overlap/duration checks, slot removal/recreation, past completion, invalid IDs, safe 404 responses and HTML confirmation. Independent code review identified the slot-recreation edge case; it was corrected and covered by a regression test. A browser check also completed a real login and booking in a separate MySQL demo database.

Run and submission

```
docker compose up -d --wait
mvn spring-boot:run

# Disposable H2 UI demo
mvn spring-boot:run -Dspring-boot.run.profiles=demo

# Fast automated suite
mvn test
```

Source, this PDF report, editable report text, setup instructions and a six-minute code-walkthrough outline accompany the delivery. The student must record the required real code-walkthrough video and provide its link. No video recording or course submission is claimed by this report. Course General Rules were not part of the attached Milestone 2 PDF. New MySQL schema initialization does not migrate an existing Milestone 1 database.

6. References and source map

Assignment: SJSU CMPE 172, Milestone 2 - Core Implementation: Booking & Concurrency, supplied two-page PDF.

Primary implementation: Lushangtu123/CMPE-172-Milestone. This report describes the Milestone 2 branch implementation, not the previous Milestone 1 main branch.

MySQL 8.4 Reference Manual, Locking Reads: https://dev.mysql.com/doc/refman/8.4/en/innodb-locking-reads.html

MySQL 8.4 Reference Manual, CREATE INDEX: https://dev.mysql.com/doc/refman/8.4/en/create-index.html (unique-index NULL semantics).

Spring Framework, Using @Transactional: https://docs.spring.io/spring-framework/reference/data-access/transaction/declarative/annotations.html

Spring Security, JDBC Authentication: https://docs.spring.io/spring-security/reference/servlet/authentication/passwords/jdbc.html

Spring Guides, Securing a Web Application: https://spring.io/guides/gs/securing-web/

Source files to explain in the video

```
controller/PageController.java
controller/AppointmentController.java
security/Account.java
security/SecurityConfig.java
service/BookingTransaction.java
service/AppointmentService.java
service/SlotService.java
repository/SlotRepository.java
repository/AppointmentRepository.java
exception/GlobalExceptionHandler.java
resources/schema-mysql.sql
BookingWorkflowTests.java / BookingServiceTests.java
TransactionTests.java
```

Java files above are under src/main/java/com/example/tutorconnect. Templates are under src/main/resources/templates. Tests are under src/test/java/com/example/tutorconnect. The README contains all API routes and the dedicated MySQL test command. All appointment times are explicitly UTC.